

AI Agent Stage 1-8 面试速记与项目深化手册

围绕 Agent loop、RAG、Harness、WorkflowState、记忆与证据链的面试表达

30 秒总述：我的项目不是只调用大模型生成文本，而是围绕“任务规划-工具执行-证据校验-反思补查-报告生成”搭了一条可追踪的 Agent loop。我主要负责上下文管理、记忆保存和 WorkflowState 设计，把异常输入、归因计划、工具结果、证据链、反思日志和结论统一沉淀，支撑复核、bad case 分析和后续归因记忆复用。

一、Stage 1-8 总览

Stage	必须会讲的核心	关键词
Stage 1 Agent Loop	observe -> think -> act -> observe ; ToolCall、ToolResult、trace、stop condition。	loop / trace
Stage 2 RAG + Memory	chunk、retrieval、citation、grounded answer、memory；先找证据再回答。	RAG / citation
Stage 3 Harness	tool registry、permission gate、session store、trace event、预算与权限边界。	harness
Stage 4 Multi-Agent	职责边界、输入输出 schema、supervisor、停止条件；不是角色聊天。	coordination
Stage 5 Skills/Protocols	Skill 是可复用流程知识；MCP 接工具/数据；A2A/ACP 管通信边界。	skill / MCP
Stage 6 Browser Agent	DOM/截图/动作日志/恢复策略；提交、支付、发布等动作必须确认。	browser safety
Stage 7 Eval/Observability	固定任务集、失败分类、trace 定位、成本/延迟/工具次数统计。	eval
Stage 8 Ship	明确用户、任务、成功标准、README、部署、日志、权限、回归测试。	production

二、Stage 1-8 深度详解

Stage 1: Minimal Agent Loop

维度	内容
学习目标	掌握一个 agent 如何在状态中观察、选择工具、执行工具、读取 observation，并在满足条件时输出 final answer。
核心概念	ToolCall、ToolResult、AgentStep、ToolRegistry、tool schema、trace、max_steps、错误停止。
工程实现	最小代码通常包含 model/policy、工具注册表、工具函数、循环控制器和 trace 输出。面试要强调：结构化 ToolCall 比自然语言动作更稳定。
项目映射	你的“任务规划-工具执行-证据校验-反思补查-报告生成”就是一个业务版 agent loop；异常输入是 observe，归因计划是 think，工具调用是 act，工具结果和证据是 observation。
常见追问	为什么需要 max_steps？工具参数错误怎么办？trace 能定位哪些问题？为什么不能让模型直接执行危险动作？

Stage 2: Tool Use, RAG, Memory

维度	内容
学习目标	掌握资料型 agent 如何先检索证据，再生成 grounded answer，并把可复用信息写入 memory。
核心概念	chunk、tokenize、retrieval、top_k、citation、grounded answer、short-term memory、long-term memory、trace vs memory。
工程实现	RAG 链路要能说清：文档加载 -> chunk -> 向量/词频检索 -> 排序 -> 证据拼接 -> 引用输出。没有证据时要拒答或降低置信度。
项目映射	你的证据链和归因记忆属于 Stage 2 能力：工具结果和归因结论需要结构化沉淀，后续 bad case 和类似异常可复用这些记忆。
常见追问	召回不准怎么办？chunk 太大/太小的影响？memory 里不该存什么？如何避免模型基于无证据内容生成结论？

Stage 3: Agent Harness

维度	内容
学习目标	理解模型外面的工程运行层：工具、权限、状态、session、trace、预算、错误恢复都由 harness 管。
核心概念	ToolRegistry、PermissionGate、SessionStore、TraceEvent、budget、sandbox、retry、approval。
工程实现	Harness.run 的典型流程是：接收任务 -> 规划 tool call -> 查 registry -> 权限检查 -> 执行工具 -> 写 trace -> 更新状态 -> 判断停止。
项目映射	WorkflowState + 工具执行 + 证据校验 + 反思日志就是你的项目 harness 思维。它让模型输出不再是黑盒，而是可审计的执行链路。
常见追问	哪些工具要人工确认？权限检查放在哪里？为什么状态不能只靠 prompt？如何用 session 复盘一次失败运行？

Stage 4: Multi-Agent Coordination

维度	内容
学习目标	理解多 agent 的难点不是角色多，而是职责边界、输入输出 schema、supervisor 控制和停止条件。
核心概念	researcher/writer/reviewer/reviser、supervisor、graph、schema、循环控制、上下文隔离。
工程实现	多 agent 要避免无限争论和上下文膨胀。每个 agent 应只做一类任务，并输出可校验结构，由 supervisor 决定是否流转。
项目映射	你的链路可映射为 planner、tool executor、evidence checker、reflector、reporter 等模块协作；即使不是独立 agent，也体现了职责分离。
常见追问	什么时候需要 multi-agent？怎么避免循环？reviewer 输出应该是什么结构？多 agent 比单 agent 多了哪些工程风险？

Stage 5: Skills, MCP, Protocols

维度	内容
学习目标	掌握能力打包：Tool 是函数，Skill 是一套可复用流程知识，MCP/A2A/ACP 是不同层面的连接协议。
核心概念	Tool、Skill、SKILL.md、template、smoke test、MCP server、A2A、ACP。
工程实现	一个好 skill 应包含适用场景、步骤、脚本/模板和验收标准。MCP 更偏连接外部工具和数据源，不等于 skill。
项目映射	归因流程、报告模板、证据校验规则、bad case 分析流程都可以沉淀成 skill，让 agent 在类似任务中稳定复用。
常见追问	Skill 和 prompt 有什么区别？Skill 和 MCP 有什么区别？如何证明一个 skill 真的提高成功率？

Stage 6: Browser and Computer-Use Agents

维度	内容
学习目标	理解浏览器/计算机使用型 agent 的观察来源、动作日志、失败恢复和权限边界。
核心概念	DOM、screenshot、click/type/navigation、action log、loading/retry、human approval。
工程实现	DOM 适合结构化信息提取，截图适合视觉校验，动作日志适合审计。提交、支付、发布、删除等动作必须人工确认。
项目映射	如果归因工具涉及后台页面、监控平台或工单系统，浏览器 agent 需要记录动作、截图/DOM 证据，并对外部影响动作设置权限门。
常见追问	DOM 和截图分别解决什么问题？页面加载失败怎么办？为什么不能让 agent 自动提交表单？

Stage 7: Evaluation, Observability, Safety

维度	内容
学习目标	掌握如何证明 agent 可靠：固定 eval、失败分类、trace 观测、安全边界和成本/延迟指标。
核心概念	eval set、expected vs actual、failure category、observability、latency、cost、tool-call count、safety guardrail。
工程实现	Eval 表至少包含任务、期望、实际、是否通过、失败分类。Trace 用来定位失败来自 prompt、工具、检索、状态、模型还是生成。
项目映射	你的 bad case 分析正对应 Stage 7：用 WorkflowState 和 trace 回看一次归因失败，定位是计划错、工具错、证据不足、反思没触发还是报告生成过度概括。
常见追问	怎么证明 agent 变好了？失败分类怎么设计？只靠人工看 demo 有什么问题？如何统计工具调用成本和延迟？

Stage 8: Ship a Real Agent

维度	内容
学习目标	把 agent 从 demo 变成可交付系统：明确用户、任务、成功标准、部署形态、日志、权限、回归测试和 README。
核心概念	user/task/success criteria、CLI/Web/API、deployment、README、reproducibility、monitoring、rollback。
工程实现	交付不是模型能答一次，而是别人能复现、能监控、能回滚、能分析失败、能控制成本和权限。
项目映射	你的报告生成链路如果要上线，需要定义异常归因成功标准、人工复核机制、bad case 回流、记忆更新策略和权限边界。
常见追问	上线后如何监控？如何回滚错误记忆？如何处理敏感数据？如何让新同事复现实验？

三、Stage 1-8 拆解记忆卡

Stage	你要会讲什么	怎么映射到你的项目
1 Loop	Agent 不是一次生成，而是带状态地观察、决策、行动、再观察。	你的链路从异常输入开始，经计划、工具、证据、反思、报告循环推进。
2 RAG/Memory	先找证据再回答；memory 沉淀历史，不等于 trace。	工具结果、归因证据和结论进入记忆，为后续归因复用。
3 Harness	工具注册、权限、状态、trace、预算由 harness 管。	WorkflowState + 工具执行 + 证据校验就是项目中的 harness 思维。
4 Multi-Agent	多 agent 不是聊天，而是职责和 schema 协调。	规划、执行、反思、报告可以看作职责分离的模块协作。
5 Skills	Skill 是可复用流程知识，Tool 是函数，MCP 接外部能力。	归因流程、报告模板、证据校验规则都可以沉淀成 skill。
6 Browser	浏览器/页面动作必须有 DOM、截图、日志和权限边界。	如果工具包含页面排查或平台操作，高风险动作必须人工确认。
7 Eval	固定测试集和失败分类比 demo 更能证明 agent 可靠。	bad case 分析可定位失败来自计划、工具、检索、反思还是生成。
8 Ship	交付要有用户、任务、成功标准、日志、权限和回归测试。	你的报告生成链路要能被复核、复现、迭代和上线监控。

四、核心概念速记

概念	面试表达
Agent loop	带状态地观察、结构化决策、安全行动、记录 trace，并在满足条件时停止。核心不是循环，而是 observation-driven decision。
Tool schema	工具说明书：name、description、parameters。模型根据 schema 生成结构化 ToolCall，程序再执行。
RAG	先检索证据，再基于证据回答。关键是 chunk、retrieval、citation 和知识不足时拒答。

概念	面试表达
Memory vs Trace	trace 记录一次运行过程，用于 debug；memory 跨运行保存历史事实、偏好、问答和可复用经验。
Harness	模型外的工程外壳，负责工具注册、权限、状态、session、trace、预算、错误处理和恢复。
WorkflowState	统一状态容器，承载输入、计划、工具结果、证据、反思决策和最终结论，是复杂 agent 可控流转的关键。

五、把知识映射到你的简历项目

简历原始表达：基于 Claude Code 辅助完成“任务规划-工具执行-证据校验-反思补查-报告生成”核心链路的 Agent loop 开发，负责上下文管理、记忆保存模块，参与设计 WorkflowState，统一承载异常输入、归因计划、工具结果和反思决策等关键上下文；同时结构化记录工具结果、归因证据链、反思日志和归因结论，为异常结论复核、bad case 分析、归因记忆沉淀提供基础。

项目技术拆解

- 1 任务规划：把异常输入转成可执行归因计划，明确要调用哪些工具、查哪些证据、怎样判断是否足够。
- 2 工具执行：将工具调用结果结构化写回 WorkflowState，而不是散落在日志或自由文本中。
- 3 证据校验：将工具结果和归因结论绑定，形成 evidence chain，避免结论没有来源支撑。
- 4 反思补查：如果证据不足、工具结果冲突或结论置信度低，触发二次查询、补充工具或改写计划。
- 5 报告生成：从状态中抽取计划、证据、反思过程和结论，生成可复核、可解释的归因报告。

WorkflowState 可以这样讲

维度	面试表达
为什么需要	长链路 agent 不能只靠 prompt 串上下文，必须有显式状态，才能稳定传递输入、计划、工具结果和反思决策。
承载什么	异常输入、归因计划、工具结果、证据链、反思日志、补查决策、最终归因结论和置信信息。
解决什么问题	避免上下文丢失、工具结果不可追踪、结论不可复核、bad case 无法归因分析。
面试落点	WorkflowState 是 agent harness 中的 state store；它让 agent loop 从“会跑”变成“可观测、可复盘、可迭代”。

六、面试高频问答

Q1：你们的 Agent loop 和普通 workflow 有什么区别？

普通 workflow 路径固定；我们的链路虽然有主流程，但是否补查、补查什么、是否接受结论，会根据工具 observation 和证据充分性动态决策。因此核心是 observation-driven decision。

Q2：你负责的上下文管理具体做了什么？

我把异常输入、归因计划、工具结果、证据链和反思决策统一到 WorkflowState，保证每一步有结构化输入输出。这样后续报告生成、结论复核和 bad case 分析都能从状态中追溯。

Q3：记忆保存和日志有什么区别？

日志/trace 记录一次运行发生了什么；memory 更偏跨任务沉淀，比如历史归因模式、典型 bad case、已验证证据和可复用经验。我的模块把可复用的归因结论和证据结构化保存，便于后续补充上下文。

Q4：如何保证归因结论不是模型胡编？

关键是 evidence chain。工具结果、证据片段、反思日志和最终结论都要结构化绑定；报告生成时只能引用状态里已经存在的证据，证据不足时触发补查或降低结论确定性。

Q5：bad case 怎么分析？

我会从 trace 和 WorkflowState 回看：是计划错、工具结果缺失、证据冲突、反思没触发，还是报告生成阶段过度概括。因为每一步都有结构化记录，所以可以定位失败来源。

Q6：Claude Code 在项目里起什么作用？

Claude Code 更像开发协作工具，辅助拆任务、生成/修改代码、补测试和迭代实现；真正的业务价值在于我们把 Agent loop、WorkflowState、工具结果和证据链工程化落地。

七、一分钟项目讲述模板

我做的是一个面向异常归因的 Agent 链路，不是简单让大模型输出结论。系统会先把异常输入转成归因计划，再调用工具获取事实证据，然后校验证据是否支持结论；如果证据不足或有冲突，会进入反思补查，最后生成带证据链的报告。我的重点工作是上下文管理和记忆保存，参与设计 WorkflowState，把异常输入、计划、工具结果、证据链、反思日志和结论统一承载。这样一方面保证每一步可追踪，另一方面也为结论复核、bad case 分析和归因记忆沉淀提供基础。

八、三分钟深入讲述模板

- 1 背景：异常归因任务天然不确定，不能只靠一次模型生成，需要根据工具结果动态补查和修正。
- 2 架构：核心链路是任务规划、工具执行、证据校验、反思补查、报告生成，形成一个可控的 Agent loop。
- 3 状态：用 WorkflowState 统一承载异常输入、归因计划、工具结果、证据链、反思日志和最终结论。
- 4 证据：每个工具结果都会结构化保存，并和归因结论建立映射，报告生成时可追溯来源。
- 5 反思：当证据不足、工具失败或结论不稳定时，反思模块会触发补查或调整计划。
- 6 价值：提升结论可解释性、复核效率和 bad case 分析效率，同时沉淀可复用的归因记忆。

九、Stage 1-8 面试追问速答

Stage	速答
Stage 1	Agent loop 的关键是 observation-driven decision；trace 用来定位 thought/action/observation 哪一步出错。
Stage 2	RAG 的价值是 grounded answer；memory 用来跨运行沉淀，不等于一次运行的 trace。
Stage 3	Harness 是模型外的工程层；权限、状态、日志、预算和恢复都应该由 harness 管。
Stage 4	Multi-agent 重点是协调：职责、schema、supervisor、停止条件，不是多角色聊天。
Stage 5	Skill 是流程知识库；Tool 是函数；MCP 是外部工具和数据源协议。
Stage 6	Browser agent 要结合 DOM、截图和动作日志，高风险网页动作必须人工确认。

Stage	速答
Stage 7	Eval 要固定任务、期望、实际、失败分类；observability 要能定位失败来源。
Stage 8	Ship 需要明确用户、任务、成功标准、权限边界、README、部署和回归测试。

十、你要重点记忆的关键词

- Agent loop：observe -> think -> act -> observe，关键是根据 observation 决定下一步。
- WorkflowState：把长链路上下文显式化，避免状态丢失和结论不可追溯。
- Evidence chain：工具结果、证据、反思和结论之间的结构化关联。
- Reflection：证据不足或冲突时触发补查，而不是直接生成最终结论。
- Memory：沉淀可复用归因经验和历史结论，不等同于日志。
- Harness：让模型动作安全、可控、可观测、可恢复的工程外壳。
- Bad case analysis：用 trace 和状态定位失败来自计划、工具、检索、反思还是生成。

十一、面试自检清单

- 能否用 30 秒讲清楚项目不是简单 prompt，而是 Agent loop？
- 能否解释 WorkflowState 里有哪些字段，以及每个字段为什么必要？
- 能否举例说明一次工具失败后如何进入反思补查？
- 能否解释 memory 和 trace 的区别？
- 能否说明证据链如何支撑结论复核？
- 能否把自己的工作和 Stage 1-8 任意一个概念对应起来？

十二、扩展面试题库

主题	可能追问与答题抓手
Agent loop	问：如果工具失败，agent 应该直接停止还是重试？答：看失败类型。参数错误可修正重试，检索不足可改写 query，权限拒绝要人工确认，达到预算则停止。
WorkflowState	问：WorkflowState 为什么不直接用一串 messages？答：messages 难以稳定承载结构化字段，WorkflowState 能让计划、工具结果、证据、反思和结论可追踪、可测试。
证据链	问：证据链怎么设计？答：每个结论要绑定来源工具、原始结果、证据片段、置信信息和反思记录，报告只能引用状态中已有证据。
反思补查	问：什么时候触发 reflection？答：工具失败、证据不足、证据冲突、结论置信度低、报告缺少关键字段或命中 bad case 规则时。
RAG	问：RAG 召回不准怎么办？答：先看 query、chunk、top_k 和 citation；再做 query rewrite、embedding/hybrid search、rerank 或补充知识库。
Memory	问：什么信息不应该进长期记忆？答：临时日志、未验证猜测、敏感信息、一次性中间状态；长期记忆应保存可复用且可信的归因经验。

主题	可能追问与答题抓手
Eval	问：怎么证明 agent 变好了？答：用固定 eval 集比较成功率、失败分类、工具调用次数、延迟和人工复核通过率，而不是只展示 demo。
安全	问：哪些动作需要人工确认？答：写文件、删除、发送消息、提交表单、发布内容、支付、修改线上配置，以及任何不可逆或影响外部系统的动作。
Claude Code	问：Claude Code 辅助开发和项目 agent 有什么区别？答：Claude Code 是开发协作工具；项目 agent 是业务系统中的可执行归因链路，两者不要混为一谈。
工程落地	问：上线后怎么排查线上 bad case？答：用 trace + WorkflowState 回放，定位失败来自计划、工具、检索、反思、记忆还是报告生成。